



VIT[®]
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

SCHOOL OF COMPUTER SCIENCE ENGINEERING
WINTER SEMESTER 2025-2026

Assessment -1

Slot: L5 + L6

Class: VL2025260504771

Programme Name & Branch: B. Tech & CSE

Course Code & Title: BCSE309P — Cryptography and Network Security (LO)

Faculty Name: Dr. K. Murugan

Maximum Marks: 10

Submitted by: Aryan Chopra

Reg. no. :23BCI0144

-
- 1. Consider a sender and receiver who need to exchange data confidentially using Asymmetric encryption. Write a C program to implements RSA Encryption and Decryption.**

CODE

```
#include <stdio.h>
```

```
typedef unsigned long long ull;
```

```
ull gcd(ull a, ull b) {  
    while (b != 0) {  
        ull t = a % b;  
        a = b;  
        b = t;  
    }  
    return a;  
}
```

```
long long ext_gcd(long long a, long long b, long long *x, long long *y) {  
    if (b == 0) {  
        *x = 1;
```

```

        *y = 0;
        return a;
    }
    long long x1, y1;
    long long g = ext_gcd(b, a % b, &x1, &y1);
    *x = y1;
    *y = x1 - (a / b) * y1;
    return g;
}

```

```

ull mod_inverse(ull e, ull phi) {
    long long x, y;
    long long g = ext_gcd(e, phi, &x, &y);
    if (g != 1)
        return 0;
    long long res = x % phi;
    if (res < 0)
        res += phi;
    return (ull)res;
}

```

```

ull mod_pow(ull base, ull exp, ull mod) {
    ull result = 1;
    base %= mod;
    while (exp > 0) {
        if (exp & 1)
            result = (result * base) % mod;
        base = (base * base) % mod;
        exp >>= 1;
    }
    return result;
}

```

```

int main() {
    ull p, q, e, msg;

    printf("Enter prime p: ");

```

```

scanf("%llu", &p);

printf("Enter prime q: ");
scanf("%llu", &q);

ull n = p * q;
ull phi = (p - 1) * (q - 1);

printf("n = %llu\n", n);
printf("phi(n) = %llu\n", phi);

printf("Enter public exponent e: ");
scanf("%llu", &e);

if (gcd(e, phi) != 1 || e <= 1 || e >= phi) {
    printf("Invalid public key\n");
    return 0;
}

ull d = mod_inverse(e, phi);

printf("Public Key (e, n) = (%llu, %llu)\n", e, n);
printf("Private Key (d, n) = (%llu, %llu)\n", d, n);

printf("Enter message (number < n): ");
scanf("%llu", &msg);

ull cipher = mod_pow(msg, e, n);
printf("Encrypted message = %llu\n", cipher);

ull decrypted = mod_pow(cipher, d, n);
printf("Decrypted message = %llu\n", decrypted);

return 0;
}

```

OUTPUT

```

Enter prime p: 11
Enter prime q: 17
n = p*q = 187
phi(n) = (p-1)(q-1) = 160
Enter public exponent e (1<e<phi and gcd(e,phi)=1): 7
Public Key (e, n) = (7, 187)
Private Key (d, n) = (23, 187)
Enter message as a number (0 <= msg < n): 88
Encrypted (ciphertext) = 11
Decrypted (original) = 88

...Program finished with exit code 0
Press ENTER to exit console.

```

ALGORITHM

Step 1: Key Generation

1. Select two large prime numbers p and q .
2. Compute $n = p \times q$.
3. Compute Euler's totient value $\phi(n) = (p - 1)(q - 1)$.
4. Choose a public exponent e such that $1 < e < \phi(n)$ and $\gcd(e, \phi(n)) = 1$.
5. Compute the private exponent d such that $d \equiv e^{-1} \pmod{\phi(n)}$.

Step 2: Encryption

6. Represent the plaintext message as an integer M such that $0 \leq M < n$.
7. Compute the ciphertext using $C = M^e \bmod n$.
8. Transmit the ciphertext C to the receiver.

Step 3: Decryption

9. Receive the ciphertext C .
10. Compute the plaintext using $M = C^d \bmod n$.
11. Output the original message M .

2. Implement Elliptic curve cryptography Encryption and Decryption.

CODE

```
#include <stdio.h>
```

```
typedef long long ll;
```

```

typedef struct {
    ll x, y;
    int inf;
} Point;

static const ll p = 9739;
static const ll a = 497;
static const ll b = 1768;
static const Point G = {1804, 5368, 0};

```

```

ll mod(ll x) {
    x %= p;
    if (x < 0) x += p;
    return x;
}

```

```

ll egcd(ll A, ll B, ll *x, ll *y) {
    if (B == 0) {
        *x = 1;
        *y = 0;
        return A;
    }
    ll x1, y1;
    ll g = egcd(B, A % B, &x1, &y1);
    *x = y1;
    *y = x1 - (A / B) * y1;
    return g;
}

```

```

ll modinv(ll k) {
    ll x, y;
    ll g = egcd(mod(k), p, &x, &y);
    if (g != 1) return -1;
    return mod(x);
}

```

```

Point point_neg(Point P) {
    if (P.inf) return P;
    Point R = {P.x, mod(-P.y), 0};
    return R;
}

```

```

Point point_add(Point P, Point Q) {

```

```

    if (P.inf) return Q;
    if (Q.inf) return P;

    if (P.x == Q.x && mod(P.y + Q.y) == 0) {
        Point R = {0, 0, 1};
        return R;
    }

    ll m;
    if (P.x == Q.x && P.y == Q.y) {
        ll num = mod(3 * (P.x * P.x % p) + a);
        ll den = modinv(mod(2 * P.y));
        if (den == -1) {
            Point R = {0, 0, 1};
            return R;
        }
        m = mod(num * den);
    } else {
        ll num = mod(Q.y - P.y);
        ll den = modinv(mod(Q.x - P.x));
        if (den == -1) {
            Point R = {0, 0, 1};
            return R;
        }
        m = mod(num * den);
    }

    ll rx = mod(m * m - P.x - Q.x);
    ll ry = mod(m * (P.x - rx) - P.y);

    Point R = {rx, ry, 0};
    return R;
}

Point scalar_mul(ll k, Point P) {
    Point R = {0, 0, 1};
    Point A = P;

    while (k > 0) {
        if (k & 1) R = point_add(R, A);
        A = point_add(A, A);
        k >>= 1;
    }
}

```

```

    return R;
}

```

```

int find_y(ll x, ll *y_out) {
    ll rhs = mod(x * x % p * x + a * x + b);
    for (ll y = 0; y < p; y++) {
        if (mod(y * y) == rhs) {
            *y_out = y;
            return 1;
        }
    }
    return 0;
}

```

```

Point encode(ll m) {
    for (ll i = 0; i < 100; i++) {
        ll x = m * 100 + i;
        if (x >= p) break;
        ll y;
        if (find_y(x, &y)) {
            Point M = {x, y, 0};
            return M;
        }
    }
    Point R = {0, 0, 1};
    return R;
}

```

```

ll decode(Point M) {
    if (M.inf) return -1;
    return M.x / 100;
}

```

```

void print_point(const char *s, Point P) {
    if (P.inf)
        printf("%s = INF\n", s);
    else
        printf("%s = (%lld, %lld)\n", s, P.x, P.y);
}

```

```

int main() {
    ll d, k, m;

```

```

printf("Enter private key d: ");
scanf("%lld", &d);

Point Q = scalar_mul(d, G);
print_point("Public Key Q", Q);

printf("Enter message m: ");
scanf("%lld", &m);

Point M = encode(m);
if (M.inf) {
    printf("Encoding failed\n");
    return 0;
}
print_point("Message Point M", M);

printf("Enter random key k: ");
scanf("%lld", &k);

Point C1 = scalar_mul(k, G);
Point C2 = point_add(M, scalar_mul(k, Q));

print_point("C1", C1);
print_point("C2", C2);

Point M_dec = point_add(C2,
point_neg(scalar_mul(d, C1)));
print_point("Decrypted Point", M_dec);

printf("Decrypted message = %lld\n",
decode(M_dec));

return 0;
}

```

OUTPUT


```
Enter private key d: 121
Public Key Q = (4249, 1038)
Enter message m: 42
Message Point M = (4200, 4268)
Enter random key k: 77
C1 = (6561, 940)
C2 = (891, 4194)
Decrypted Point = (4200, 4268)
Decrypted message = 42

...Program finished with exit code 0
Press ENTER to exit console.
```

ALGORITHM

Step 1: Curve and Parameter Selection

1. Select a prime number p and define the elliptic curve $y^2 = x^3 + ax + b \pmod{p}$.
2. Choose a base point G on the elliptic curve.

Step 2: Key Generation

3. Select a private key d (a random positive integer).
4. Compute the public key $Q = dG$.

Step 3: Encryption

5. Represent the plaintext message as a point M on the elliptic curve.
6. Select a random integer k .
7. Compute $C_1 = kG$.
8. Compute $C_2 = M + kQ$.
9. Transmit the ciphertext pair (C_1, C_2) .

Step 4: Decryption

10. Receive the ciphertext pair (C_1, C_2) .
11. Compute $dC_1 = d(kG)$.
12. Recover the message point using $M = C_2 - dC_1$.
13. Decode the point M to obtain the original plaintext.